

Vectors and Embeddings: Meaning as Coordinates

FRIDAY AI Education — Episode 013 · Printable Companion Paper

Abstract

Computers do not read. They compute. For decades this meant that a machine could match the *word* "revenue" but had no idea that "topline," "sales," and "ARR" were circling the same thing. Embeddings are the technique that closed most of that gap. An embedding turns a piece of text — a word, a sentence, a paragraph, a whole document — into a list of numbers that encodes its *meaning* well enough that similar meanings end up near each other in space. Once meaning has coordinates, "find me things like this" becomes arithmetic, and arithmetic is something computers are extraordinarily good at.

This paper explains what an embedding actually is, the mechanism that produces it, and the machinery — vector databases, nearest-neighbor search, chunking — that makes it useful at the scale of a company's knowledge. It uses three running examples: searching a personal knowledge base (GBrain), pulling the right clip out of a library of podcasts, and running semantic search over company documents. It ends where every honest treatment of this topic should end: with the limits. Embeddings encode *similarity*, not *truth*. They are a retrieval tool, not a judgment. Knowing the difference is the part that separates someone who can deploy this in a serious company from someone who gets burned by it.

1. Why this is worth an hour of your attention

Most of the leverage in an AI-native company does not come from the model being clever. It comes from the model being *given the right context at the right moment*. A large language model with no access to your documents is a brilliant consultant who has never seen your business. The same model, handed the three most relevant paragraphs from your own archive, becomes something far more useful — and far more defensible in a room full of skeptics.

The bridge between "your archive" and "the three most relevant paragraphs" is built almost entirely out of embeddings. When FRIDAY answers a question by retrieving from GBrain, embeddings are doing the retrieving. When you imagine a product that makes a person's entire public body of work searchable, citable, and operationally useful, embeddings are the load-bearing wall. This is not a peripheral technique you can delegate forever. It is one of the half-dozen ideas you need in your own head to tell the difference between a real capability and a demo that will fall apart in production.

The good news: the core idea is genuinely simple, and it survives being explained without mathematics. The discipline is in respecting what it *cannot* do.

2. The core idea: meaning as coordinates

Start with a map. On a map, every city is a pair of numbers — latitude and longitude. Two numbers are enough to say where a place is, and crucially, to say which places are *near* each other. Paris and Brussels are close; Paris and Tokyo are not. You can answer "what's near Paris?" purely by comparing coordinates, without knowing anything

about the cities themselves.

An embedding does the same thing for *meaning* instead of geography. It assigns every piece of text a position — not in two dimensions, but in hundreds or thousands. A modern embedding might place each sentence at a point described by 1,536 numbers. That list of numbers is called a **vector**. You cannot picture 1,536 dimensions, and you don't need to. The only property that matters is the one the map already gave you: **things with similar meaning land in nearby positions**.

So the sentence "Our churn rate climbed in Q3" and the sentence "We lost more customers than usual last quarter" share almost no words. A keyword system sees two unrelated strings. An embedding system places them close together, because it has learned that they *mean* nearly the same thing. That closeness is the whole game. We call it **semantic similarity** — similarity of meaning — and we measure it as a distance (or angle) between vectors. Note the word *semantic*. This has nothing to do with geographic distance and everything to do with conceptual distance; the spatial language is a metaphor we get to keep because the math behaves spatially.

One line worth memorizing: **an embedding is a translation of meaning into coordinates, chosen so that distance means dissimilarity**.

3. The mechanism: where do the coordinates come from?

A fair question at this point: who decides that "churn" and "lost customers" belong near each other? Nobody hand-assigns these positions. They are *learned* from how language is actually used, and the learning rests on one deceptively powerful observation, sometimes called the distributional hypothesis:

A word is known by the company it keeps.

Words that show up in similar contexts tend to mean similar things. "Coffee" and "tea" appear around the same neighbors — *cup, morning, hot, drink* — so a system that reads enough text notices they behave alike, and places them near each other. It never needs a dictionary. It just needs a great deal of text and a way to adjust positions until words that travel in the same crowds end up in the same region.

Jay Alammar's *Illustrated Word2vec* (in the reading guide) shows the classic early version of this with pictures, and it is worth seeing once. The famous demonstration is that the learned coordinates capture *relationships*, not just clusters. The vector arithmetic $\text{king} - \text{man} + \text{woman}$ lands very close to queen. The direction you travel to get from "man" to "woman" is roughly the same direction that takes you from "king" to "queen." The model was never taught about royalty or gender. The structure fell out of the statistics of usage. That result is what convinced the field that these coordinates were encoding something real about meaning, not just bucketing synonyms.

Modern embedding models are far more capable than Word2vec in one decisive way: they are **context-sensitive**. The word "bank" gets a different position in "river bank" than in "investment bank," because the model reads the surrounding words before placing it. This is why today's embeddings can handle whole sentences and paragraphs, not just isolated words — the model considers the entire passage and produces a single vector summarizing its meaning. You do not train these yourself. You call a model — OpenAI's embeddings API is the canonical example in the reading guide — hand it text, and receive a vector back. That is the entire interface from your side: **text in, vector out**. The sophistication is sealed behind the call.

A small but clarifying detail: the individual numbers in the vector are not human-readable. Coordinate #847 does not mean "formality" or "is about money." The meaning is distributed across all the dimensions at once, in a way no person interprets directly. You never inspect the coordinates. You only ever compare them.

4. From one vector to a working system

A single embedding is inert. The value appears when you have many of them and want to find the relevant few. Three pieces of machinery turn embeddings into search.

4.1 Chunking: cutting documents to the right size

You rarely embed a whole document as one vector. A forty-page strategy memo crushed into a single point loses all its internal structure — the one sharp paragraph about pricing gets averaged together with thirty-nine pages of context until it means almost nothing. So before embedding, you split source material into smaller passages. This is **chunking**.

A chunk is typically a few hundred words — a paragraph, a section, a slide, a couple of minutes of transcript. Each chunk gets its own embedding. Chunking is unglamorous and quietly decisive: chunk too large and your results are vague; chunk too small and each piece loses the context that made it meaningful. ("It tripled" is useless without knowing what *it* is.) Much of the difference between a retrieval system that feels sharp and one that feels muddy comes down to chunking decisions that never appear in any demo. It is the part teams most often get wrong, precisely because it looks trivial.

4.2 The vector database: storage built for comparison

Once you have thousands or millions of chunk-vectors, you need somewhere to keep them that is organized for the one operation you'll perform constantly: *given a new vector, find the closest stored ones*. A **vector database** is storage specialized for exactly that. An ordinary database is built to find exact matches — the row where `customer_id = 4471`. A vector database is built to find *nearest neighbors* in high-dimensional space, fast, even across millions of entries.

4.3 Nearest-neighbor search: the actual lookup

Here is the full motion of a search, the loop that underlies nearly every "ask your documents" feature you will ever build:

1. The user asks a question in plain language.
2. The question is embedded into a vector — the same translation applied to the query.
3. The vector database finds the stored chunk-vectors *nearest* to it.
4. Those nearest chunks are the search results.

Step 3 is **nearest-neighbor search**. At scale, the database doesn't check every vector one by one; it uses clever indexing to find *approximate* nearest neighbors — close enough, far faster. That approximation is usually an excellent trade and worth knowing exists, because it explains why results can occasionally miss something that was, technically, just slightly closer.

Notice what this design buys you: the user never has to guess the right keyword. They ask in their own words; the system matches on *meaning*. Someone searching "how do we handle unhappy customers" finds the document titled "Refund and Escalation Policy" even with zero shared words. That is the leap over keyword search, and it is the reason this technique took over.

Concept	One-line role	Failure if you skip it
Embedding	Turns text into a meaning-vector	No way to compare meaning, only exact words
Vector	The list of numbers itself	— (it's the unit)
Semantic similarity	Closeness of two vectors = closeness of meaning	Can't rank what's "relevant"
Chunking	Splits documents into right-sized passages	Results vague, or stripped of context
Vector database	Stores vectors, organized for nearest-neighbor lookup	Search too slow at real scale
Nearest-neighbor search	Finds the closest vectors to a query	No retrieval at all

5. Three business examples

5.1 GBrain search

GBrain is the knowledge and memory substrate underneath FRIDAY — the accumulated notes, decisions, documents, and history. When you ask FRIDAY something and it answers from your own material, the path is the one above: your question becomes a vector, GBrain's vector database returns the nearest chunks, and those chunks are handed to the language model as context to reason over. The model supplies the fluency; the embeddings supply the *relevance*. Without retrieval, FRIDAY is a stranger with a good vocabulary. With it, FRIDAY is operating on your actual record. This pattern — retrieve relevant context, then generate an answer grounded in it — is common enough to have a name: **RAG**, retrieval-augmented generation. Embeddings are the "retrieval" half.

5.2 Finding the right podcast clip

You have a library of recorded conversations and you want the two minutes where someone explained, say, why a particular market was mispriced — but you don't remember the words they used. Transcribe each episode, chunk the transcripts into short passages, embed each chunk. Now "where did we talk about the market being mispriced?" embeds into a vector that lands near the passage where it was discussed, *even if the speaker said "the price didn't reflect reality."* You get the timestamp without remembering the phrasing. This is the same machinery as GBrain search, pointed at audio-derived text — a direct preview of making someone's spoken body of work searchable and citable.

5.3 Semantic search over company documents

A company accumulates contracts, policies, memos, specs, decks. New hires can't find anything; experienced people half-remember where things are. Embed it all and search becomes meaning-based: "what's our policy on remote work for contractors?" surfaces the right clause regardless of how that clause is worded. The strategic point is that this turns a dormant archive into an operational asset — exactly the wedge in making a body of work *searchable, reusable, cited, and operationally useful*. The archive was always valuable; it just wasn't reachable. Embeddings make it reachable.

6. The limitation that matters most: similarity is not truth

Everything above sells the technique. This section is the one to annotate hardest, because it is where careful operators are separated from careless ones.

An embedding tells you what is *similar*. It does not tell you what is *true, current, or correct*. Those are different questions, and the system has no idea it isn't answering them.

A few specific ways this bites:

- **Confidently retrieving the wrong-but-similar thing.** Ask about your *current* refund policy and the nearest chunk might be last year's *deprecated* policy — semantically almost identical, factually wrong. The vector distance is small; the business consequence is not. The system has no concept of "outdated." It only knows "close."
- **Plausible neighbors that miss the point.** Two passages can be about the same topic while making opposite claims. "We should raise prices" and "we should not raise prices" are near-neighbors in embedding space — same subject, same vocabulary, opposite meaning. Retrieval surfaces relevance, not agreement.
- **Garbage chunks, garbage results.** If chunking split a critical caveat away from the claim it qualified, retrieval can return a confident statement and silently leave its "...unless the contract predates 2024" behind. The retrieval looked perfect.
- **Approximation gaps.** Because large-scale search is *approximate*, it can occasionally skip a result that was genuinely the best match. Usually fine; worth knowing when the stakes are high.

The deeper point: embeddings are a **retrieval** tool, and retrieval is only the first step. What you retrieve still has to be *judged* — by the language model reasoning over it, by evaluations (**evals**: the tests you run to measure whether the system is actually returning good results), by citation back to sources so a human can check, and, where the stakes warrant it, by a human approval gate before anything acts on the result. A retrieval system without evals is a system whose quality you are simply asserting. In a serious company, "trust me, the search is good" is not an answer.

This is also why an embedding-powered system is not an autonomous employee. It is a very good librarian: it finds you the relevant pages with uncanny speed, including pages you didn't know to ask for. It does not *vouch* for them. The judgment — is this current, is this right, does this actually answer the question — remains the job of the reasoning layer above it and, ultimately, of a person who can be held accountable. Build the librarian. Do not mistake it for the lawyer.

7. Vocabulary

- **Embedding** — A list of numbers representing a piece of text's meaning, arranged so that similar meanings produce nearby lists. Produced by calling an embedding model: text in, vector out.
- **Vector** — The list of numbers itself; a point in a high-dimensional space. The unit you store and compare.
- **Semantic similarity** — How close two pieces of text are *in meaning*, measured as the distance (or angle) between their vectors. "Semantic" = of meaning; nothing to do with geography.
- **Chunking** — Splitting a document into smaller passages before embedding, so each vector captures something specific rather than an average of everything.
- **Vector database** — Storage specialized for holding many vectors and finding the nearest ones to a query, fast.
- **Nearest-neighbor search** — The operation of finding the stored vectors closest to a query vector. At scale, usually *approximate* (close enough, much faster).

- **RAG (retrieval-augmented generation)** — Retrieving relevant context with embeddings, then having a language model generate an answer grounded in that context. *Not* "RAIG."
- **Evals (evaluations)** — The tests you run to measure whether a retrieval/answer system is actually returning good results. *Not* "evils." The thing that converts "I think it works" into "I measured that it works."

8. Reading guide

Two sources, read in this order. Together they are roughly an evening — not a course.

1. Jay Alammar — *The Illustrated Word2vec* · <https://jalammar.github.io/illustrated-word2vec/>

The clearest visual explanation of how words become vectors and why those vectors capture relationships. Read it for *intuition*, not implementation. The single thing to take away: the king – man + woman ≈ queen result, and the realization that nobody programmed that — it emerged from usage. Don't get lost in the training mechanics; you will never train one of these by hand. (~30–40 min.)

2. OpenAI — *Embeddings guide* · <https://platform.openai.com/docs/guides/embeddings>

The practitioner's view: how you actually obtain embeddings in a real system, and the canonical use cases (search, clustering, recommendations). Read it for the *interface and the shape of real usage* — text in, vector out, then compare. You're reading to understand what's involved in standing one up and where the work actually is (hint: chunking and evals), not to memorize API parameters. (~20–30 min.)

If you only do one thing: read Alammar for the *why*, skim OpenAI for the *how*, and re-read Section 6 of this paper for the *when not to trust it*.

9. Walking questions

Carry these on the next walk. They are diagnostic, not rhetorical.

1. Which part of "meaning as coordinates" could you now explain, cleanly, to a smart non-technical partner — without a whiteboard?
2. Which part still feels blurry? (A good candidate: *why* similar usage produces similar coordinates, or what "approximate" nearest-neighbor actually trades away.)
3. Where exactly does this already live in FRIDAY or GBrain — and at which step would a failure (stale chunk, bad split) actually hurt?
4. Name **one** bounded, low-stakes workflow where you could test embedding-based search safely — somewhere a wrong-but-similar result is annoying, not dangerous — with a human reading the results before anything acts on them. What would "good results" mean there, and how would you *measure* it rather than assert it?

10. One-sentence takeaway

Embeddings turn meaning into coordinates so machines can find what's *relevant* — but *relevant* is not the same as *true*, and the judgment of what to trust always stays one layer above the search.